

# **SE3020**

## **Distributed Systems**



### **Assignment**

#### **Group number: SE-73**

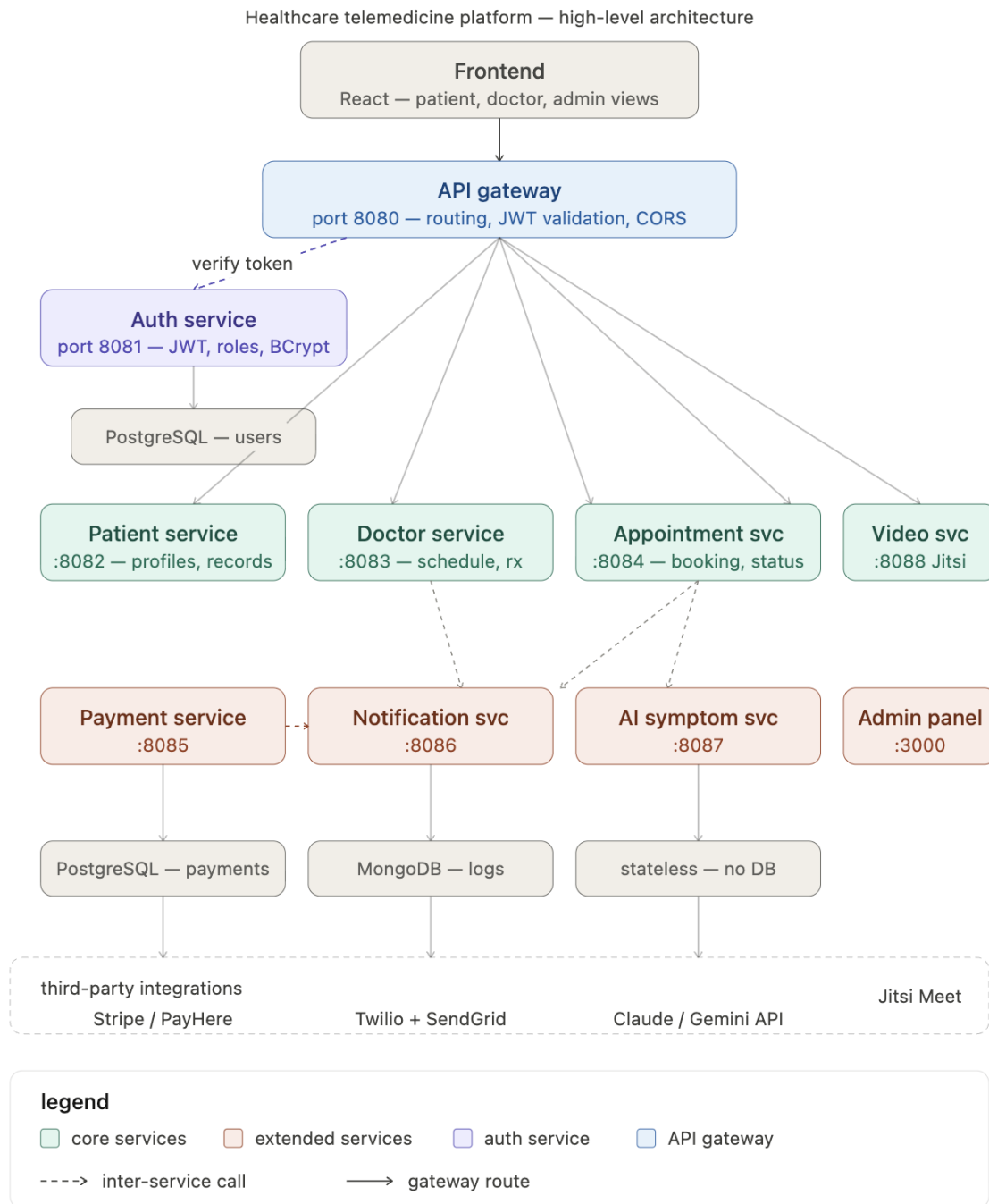
| <b>Registration number</b> | <b>Name</b>            |
|----------------------------|------------------------|
| IT23158986                 | L W P P Uthsara        |
| IT23166042                 | L A Y R Samaraweera    |
| IT23168336                 | C S B D Chandrathilaka |
| IT23292604                 | K P Athukorala         |

**Submission date: 2026-04-17**

## Table of Contents

|                                                                                          |    |
|------------------------------------------------------------------------------------------|----|
| High-level Architectural Diagram.....                                                    | 3  |
| Service Interfaces.....                                                                  | 4  |
| Patient Service - Exposed Interfaces (Port 8082) .....                                   | 4  |
| Doctor Service - Exposed Interfaces (Port 8083) .....                                    | 7  |
| Appointment Service - Exposed Interfaces (Port 8084) .....                               | 11 |
| AI Symptom Checker Service - Exposed Interfaces (Port 8087) .....                        | 15 |
| Payment Service - Exposed Interfaces (Port 8086) .....                                   | 16 |
| Workflows.....                                                                           | 17 |
| Appointment & Telemedicine Workflow (IT23158986 Uthsara L W P P).....                    | 17 |
| Payment and Notification Workflow (IT23292604 Athukorala K P) Payment Service Flow ..... | 18 |
| Doctor service workflow: IT23166042 – L A Y R Samaraweera .....                          | 19 |
| Patient Service Workflow (IT23168336 C S B D Chandrathilaka) .....                       | 22 |
| Authentication & Security Mechanisms .....                                               | 26 |
| 1. Overall Security Architecture .....                                                   | 26 |
| 2. Core Security Components .....                                                        | 26 |
| 3. Service-to-Service Security .....                                                     | 27 |
| 4. Technical Specifications .....                                                        | 28 |
| Individual contribution.....                                                             | 29 |
| Appendix.....                                                                            | 30 |
| Appointment Service .....                                                                | 30 |
| Doctor Service .....                                                                     | 39 |
| Patient service .....                                                                    | 44 |
| Telemedicine Service .....                                                               | 53 |
| Payment service .....                                                                    | 61 |
| Notification Service .....                                                               | 64 |
| AI Symptom Checker Service .....                                                         | 66 |

# High-level Architectural Diagram



# Service Interfaces

## Patient Service - Exposed Interfaces (Port 8082)

### Patient Profile Management

#### POST /api/patients

Headers: Authorization: Bearer {token}

Request:

```
{  
  "firstName": "John",  
  "lastName": "Doe",  
  "dateOfBirth": "1990-05-15",  
  "gender": "MALE",  
  "bloodGroup": "O+",  
  "phone": "+1234567890",  
  "address": "123 Main St",  
  "city": "New York",  
  "state": "NY",  
  "zipCode": "10001",  
  "emergencyContactName": "Jane Doe",  
  "emergencyContactPhone": "+1987654321"  
}
```

Response: 201 CREATED

```
{ "patientId": 1, "username": "johndoe", "email": "john@example.com" }
```

#### GET /api/patients/me

Headers: Authorization: Bearer {token}

Response: 200 OK

```
{  
  "id": 1,
```

```
"username": "johndoe",  
"firstName": "John",  
"lastName": "Doe",  
"email": "john@example.com",  
"phone": "+1234567890",  
"dateOfBirth": "1990-05-15",  
"gender": "MALE",  
"bloodGroup": "O+",  
"address": "123 Main St",  
"city": "New York"  
}
```

PUT /api/patients/me

Headers: Authorization: Bearer {token}

Request: { updated fields... }

Response: 200 OK

{ updated patient profile... }

Medical History

GET /api/patients/{id}/history

Headers: Authorization: Bearer {token}

Response: 200 OK

```
[  
  {  
    "appointmentId": 1,  
    "doctorName": "Dr. Smith",  
    "appointmentDate": "2026-04-15",  
    "diagnosis": "Common Cold",  
    "notes": "Advised rest and fluids"  
  }  
]
```

GET /api/patients/{id}/prescriptions

Headers: Authorization: Bearer {token}

Response: 200 OK

```
[  
  {  
    "prescriptionId": 1,  
    "medication": "Aspirin",  
    "dosage": "500mg",  
    "frequency": "3 times daily",  
    "duration": "7 days",  
    "prescribedBy": "Dr. Smith"  
  }  
]
```

Medical Reports Upload/Download

POST /api/patients/{id}/reports

Headers: Authorization: Bearer {token}

Form Data: file (multipart), description (optional)

Response: 201 CREATED

```
{  
  "reportId": 1,  
  "fileName": "blood_test_report.pdf",  
  "uploadedAt": "2026-04-17T20:00:00Z",  
  "downloadUrl": "/api/patients/1/reports/1/download"  
}
```

GET /api/patients/{id}/reports

Headers: Authorization: Bearer {token}

Response: 200 OK

```
[
```

```
{ "reportId": 1, "fileName": "blood_test.pdf", "uploadedAt": "..." },  
{ "reportId": 2, "fileName": "xray_report.pdf", "uploadedAt": "..." }  
]
```

GET /api/patients/{id}/reports/{reportId}

Headers: Authorization: Bearer {token}

Response: 200 OK

```
{ "reportId": 1, "fileName": "blood_test.pdf", "size": 2048000 }
```

GET /api/patients/{id}/reports/{reportId}/download

Headers: Authorization: Bearer {token}

Response: 200 OK (file download)

Content-Type: application/pdf

[binary file content]

DELETE /api/patients/{id}/reports/{reportId}

Headers: Authorization: Bearer {token}

Response: 204 NO CONTENT

## Doctor Service - Exposed Interfaces (Port 8083)

Doctor Profile Management

POST /api/doctors

Headers: Authorization: Bearer {token} (requires DOCTOR role)

Request:

```
{  
  "firstName": "John",  
  "lastName": "Smith",  
  "specialization": "Cardiology",  
  "licenseNumber": "MD123456",  
}
```

## DS Assignment

```
"experience": 15,  
"phone": "+1234567890",  
"bio": "Experienced cardiologist...",  
"education": "MD from Harvard Medical School"  
}
```

Response: 201 CREATED

```
{ "doctorId": 1, "username": "drsmith", "specialization": "Cardiology" }
```

GET /api/doctors

Query Params: ?specialization=Cardiology&page=0&size=10

Response: 200 OK

```
{  
  "doctors": [  
    {  
      "doctorId": 1,  
      "firstName": "John",  
      "lastName": "Smith",  
      "specialization": "Cardiology",  
      "experience": 15,  
      "rating": 4.8,  
      "consultationFee": 50.00  
    },  
  ],  
  "totalElements": 245  
}
```

GET /api/doctors/{id}

Response: 200 OK

```
{  
  "doctorId": 1,
```



```
"firstName": "John",  
"lastName": "Smith",  
"specialization": "Cardiology",  
"licenseNumber": "MD123456",  
"experience": 15,  
"phone": "+1234567890",  
"bio": "...",  
"rating": 4.8,  
"consultationFee": 50.00,  
"totalAppointments": 500  
}
```

PUT /api/doctors/{id}

Headers: Authorization: Bearer {token}

Request: { updated fields... }

Response: 200 OK

{ updated doctor profile... }

## Availability Management

GET /api/doctors/{id}/availability

Query Params: ?date=2026-04-20

Response: 200 OK

```
{  
  "date": "2026-04-20",  
  "availableSlots": [  
    { "slotId": 1, "startTime": "09:00", "endTime": "09:30" },  
    { "slotId": 2, "startTime": "09:30", "endTime": "10:00" }  
  ]  
}
```

## POST /api/doctors/{id}/availability

Headers: Authorization: Bearer {token} (doctor only)

Request:

```
{  
  "dayOfWeek": "MONDAY",  
  "startTime": "09:00",  
  "endTime": "17:00",  
  "slotDuration": 30  
}
```

Response: 201 CREATED

```
{ "availabilityId": 1, "status": "created" }
```

## PUT /api/doctors/{id}/availability/{availabilityId}

Headers: Authorization: Bearer {token}

Request: { updated fields... }

Response: 200 OK

## DELETE /api/doctors/{id}/availability/{availabilityId}

Headers: Authorization: Bearer {token}

Response: 204 NO CONTENT

## Ratings & Review

### POST /api/doctors/{id}/reviews

Headers: Authorization: Bearer {token}

Request:

```
{  
  "rating": 5,  
  "comment": "Excellent service!",  
  "appointmentId": 1  
}
```

```
}
```

Response: 201 CREATED

```
{ "reviewId": 1, "rating": 5 }
```

GET /api/doctors/{id}/reviews

Query Params: ?page=0&size=10&sortBy=rating

Response: 200 OK

```
[
```

```
{ "reviewId": 1, "patientName": "John", "rating": 5, "comment": "..." }
```

```
]
```

## Appointment Service - Exposed Interfaces (Port 8084)

POST /api/appointments

Headers: Authorization: Bearer {token}

Request:

```
{
```

```
  "patientId": 1,
```

```
  "doctorId": 1,
```

```
  "appointmentDate": "2026-04-20",
```

```
  "appointmentTime": "10:00",
```

```
  "consultationType": "ONLINE|IN_PERSON",
```

```
  "reason": "Regular checkup"
```

```
}
```

Response: 201 CREATED

```
{
```

```
  "appointmentId": 1,
```

```
  "status": "CONFIRMED",
```

```
  "appointmentDate": "2026-04-20",
```

```
  "appointmentTime": "10:00",
```

```
  "consultationFee": 50.00
```

```
}
```

GET /api/appointments/{id}

Headers: Authorization: Bearer {token}

Response: 200 OK

```
{  
  "appointmentId": 1,  
  "patientId": 1,  
  "doctorId": 1,  
  "patientName": "John Doe",  
  "doctorName": "Dr. Smith",  
  "appointmentDate": "2026-04-20",  
  "appointmentTime": "10:00",  
  "status": "CONFIRMED",  
  "consultationType": "ONLINE",  
  "reason": "Regular checkup"  
}
```

GET /api/appointments/patient/{patientId}

Headers: Authorization: Bearer {token}

Query Params: ?status=CONFIRMED&sort=appointmentDate,desc

Response: 200 OK

```
{  
  "appointments": [  
    { "appointmentId": 1, "doctorName": "Dr. Smith", "date": "2026-04-20" }  
  ],  
  "total": 5  
}
```

GET /api/appointments/doctor/{doctorId}

Headers: Authorization: Bearer {token}

Query Params: ?date=2026-04-20

Response: 200 OK

```
[
  {
    "appointmentId": 1,
    "patientName": "John Doe",
    "time": "10:00",
    "status": "CONFIRMED"
  }
]
```

### Appointment Rescheduling

PATCH /api/appointments/{id}/reschedule

Headers: Authorization: Bearer {token}

Request:

```
{
  "newDate": "2026-04-21",
  "newTime": "11:00"
}
```

Response: 200 OK

```
{ "appointmentId": 1, "status": "RESCHEDULED", "newDate": "2026-04-21" }
```

POST /api/appointments/{id}/cancel

Headers: Authorization: Bearer {token}

Request: { "reason": "Patient unavailable" }

Response: 200 OK

```
{ "appointmentId": 1, "status": "CANCELLED", "refundAmount": 50.00 }
```

GET /api/appointments/{id}/status

Response: 200 OK

```
{ "status": "CONFIRMED", "lastUpdated": "2026-04-17T20:00:00Z" }
```

POST /api/appointments/{id}/confirm

Headers: Authorization: Bearer {token}

Response: 200 OK

```
{ "appointmentId": 1, "status": "CONFIRMED" }
```

POST /api/symptoms/analyze

Headers: Authorization: Bearer {token}

Request:

```
{  
  "symptoms": ["headache", "fever", "cough"],  
  "duration": "3 days",  
  "severity": "MODERATE",  
  "ageGroup": "ADULT",  
  "additionalInfo": "No recent travel"  
}
```

Response: 200 OK

```
{  
  "conditions": [  
    {  
      "name": "Common Cold",  
      "probability": 85,  
      "description": "Viral respiratory infection",  
      "severity": "mild",  
      "characteristics": ["cough", "fever", "headache"]  
    }  
  ],  
  "recommendedSpecialties": ["General Practitioner", "ENT"],  
  "warnings": [],  
  "nextSteps": "Rest and stay hydrated. See doctor if symptoms persist.",  
  "confidence": "HIGH",  
  "disclaimer": "This is preliminary analysis only...",  
  "analysisTimestamp": "2026-04-17T20:00:00Z"
```

```
}
```

## AI Symptom Checker Service - Exposed Interfaces (Port 8087)

POST /api/symptoms/analyze

Headers: Authorization: Bearer {token}

Request:

```
{  
  "symptoms": ["headache", "fever", "cough"],  
  "duration": "3 days",  
  "severity": "MODERATE",  
  "ageGroup": "ADULT",  
  "additionalInfo": "No recent travel"  
}
```

Response: 200 OK

```
{  
  "conditions": [  
    {  
      "name": "Common Cold",  
      "probability": 85,  
      "description": "Viral respiratory infection",  
      "severity": "mild",  
      "characteristics": ["cough", "fever", "headache"]  
    }  
  ],  
  "recommendedSpecialties": ["General Practitioner", "ENT"],  
  "warnings": [],  
  "nextSteps": "Rest and stay hydrated. See doctor if symptoms persist.",  
  "confidence": "HIGH",  
}
```

```
"disclaimer": "This is preliminary analysis only...",  
"analysisTimestamp": "2026-04-17T20:00:00Z"  
}
```

## **Payment Service - Exposed Interfaces (Port 8086)**

*(Already provided previously)*

POST /api/payments/initiate

POST /api/payments/confirm

GET /api/payments/transaction/{transactionId}

GET /api/payments/patient/{patientId}/history

POST /api/payments/refund/{transactionId}

GET /api/payments/refund/{refundId}

GET /api/payments/admin/all

GET /api/payments/admin/statistics

POST /webhooks/stripe

GET /actuator/health



# Workflows

A brief explanation of each workflow used in the system.

## Appointment & Telemedicine Workflow (IT23158986 Uthsara L W P P)

The Appointment Service and Telemedicine Service work together to manage the lifecycle of a patient's consultation, from the initial booking to the virtual meeting.

### 1. Appointment Service Workflow

The Appointment Service is the central orchestrator for bookings and maintains the state of all medical consultations.

Booking Initiation (POST /api/appointments):

- **Validation:** The service first checks if the requested slotId is already taken by a non-cancelled appointment to prevent double-booking.
- **Persistence:** It creates an Appointment record with a default status of PENDING and a payment status of PENDING.
- **Inter-Service Sync:** It immediately calls the doctor-service (via OpenFeign) to mark the selected time slot as inactive. This ensures the slot disappears from the doctor's available list in real-time.

Payment Synchronization (RabbitMQ):

- The service listens to a RabbitMQ queue (payment\_queue).
- When the payment-service successfully processes a transaction, it publishes an event.
- The PaymentEventListener in the Appointment Service consumes this event and updates the appointment's paymentStatus to PAID automatically.

Status Management:

- Doctors or Admins can update the status (e.g., to CONFIRMED or COMPLETED).
- If an appointment is Cancelled, the service triggers a call back to the doctor-service to mark the time slot as active (available) again, allowing other patients to book it.

Retrieval: Provides dedicated endpoints to filter appointments by patientId (for the patient dashboard) or doctorUsername (for the doctor's schedule).

### 2. Telemedicine Service Workflow

This service is specifically dedicated to handling the technical requirements of Virtual (Tele) consultations using the Agora SDK.

Session Creation:

- Triggered when a virtual appointment is ready.
- It calls the appointment-service to "assert" that the appointment is confirmed before allowing a session to be created.
- It creates a virtual "Room Name" based on the appointment ID (e.g., consultation-[appointment-id]).

### Secure Access (Token Generation):

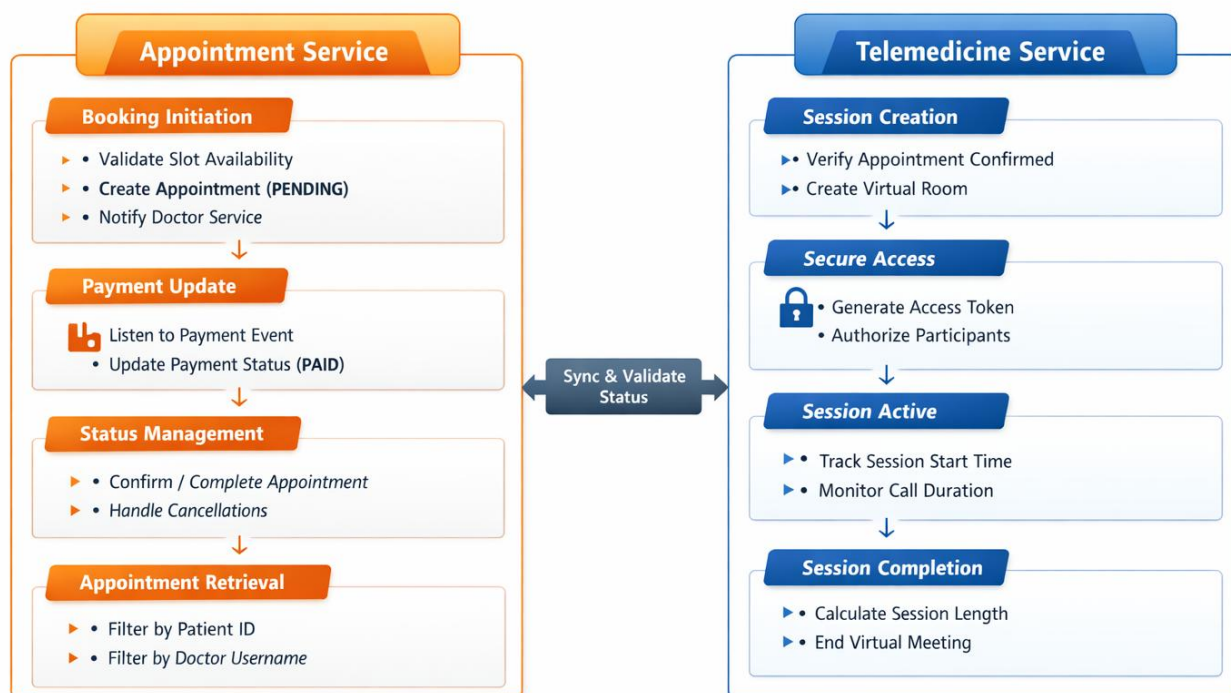
- To join a video call, the frontend requests a token from `/api/telemedicine/session/{id}/token`.
- The service uses the Agora RTC SDK to generate a dynamic, time-limited token. This ensures that only authorized participants (the specific doctor and patient) can enter the video room.

### Lifecycle Tracking:

- **Active State:** When the first participant joins and "starts" the session, the service records the `startedAt` timestamp.
- **Completion:** When the session ends, it calculates the total duration in seconds. This data can later be used for billing or quality audits.

**Persistence:** Currently, the service uses an in-memory `ConcurrentHashMap` to manage active sessions, prioritizing high-speed access for real-time signaling.

## Appointment & Telemedicine Workflow



## Payment and Notification Workflow (IT23292604 Athukorala K P)

### Payment Service Flow

- Patient initiates payment request
- API Gateway validates JWT token

- Payment Service validates input data (amount, currency, patient, doctor info)
- Creates payment intent with Stripe/PayPal
- Stores transaction record in database
- Patient confirms payment in payment provider
- Payment status verified and marked COMPLETED
- Triggers async HTTP POST to Notification Service

#### Notification Service Flow

- Receives payment confirmation from Payment Service
- Validates notification data (email, amount, etc.)
- Creates notification record in database
- Selects delivery channels (Email, SMS, Push, In-App)
- Renders email template with patient data
- Sends via all configured channels simultaneously
- Updates delivery status in database
- Creates audit trail entry

## **Doctor service workflow: IT23166042 – L A Y R Samaraweera**

### **Workflow 1: Doctor Profile Management**

Files:

Controller: DoctorProfileController.java

Service: DoctorProfileService.java

Entity: DoctorProfile.java

Flow:

Read all doctors: GET /api/doctors

Read one profile: GET /api/doctors/{doctorUsername}/profile

Read own profile: GET /api/doctors/profile (requires doctor context)

Upsert profile: PUT /api/doctors/{doctorUsername}/profile or /api/doctors/profile

Delete profile: DELETE /api/doctors/{doctorUsername}

Rules:

Admin can edit any doctor profile.

Doctor can edit only own profile.

On first upsert, profile is created with defaults (verified=false, rating=0).

## **Workflow 2: Availability Scheduling**

Files:

Controller: DoctorAvailabilityController.java

Service: DoctorAvailabilityService.java

Entity: DoctorAvailabilitySlot.java

Flow:

Get all active slots (patient/doctor): GET /api/doctors/availability/all

Get own slots: GET /api/doctors/availability

Replace full schedule: PUT /api/doctors/availability

Add one slot: POST /api/doctors/availability/slots

Delete slot: DELETE /api/doctors/availability/slots/{slotId}

Rules:

Day/start/end required.

start < end enforced.

Overlap checks enforced both:

against existing slots on add

inside batch on replace

Replace flow deletes old doctor slots and saves new set.

## **Workflow 3: Appointment Decision (Approve/Reject)**

Files:

Controller: DoctorAppointmentController.java

Service interface: DoctorAppointmentService.java

Current impl: InMemoryDoctorAppointmentService.java

Flow:

List doctor appointments: GET /api/doctors/{doctorUsername}/appointments

Accept appointment: PUT /api/appointments/{appointmentId}/accept

Reject appointment: PUT /api/appointments/{appointmentId}/reject

#### **Workflow 4: Prescription Issuance**

Files:

Controller: PrescriptionController.java

Service: PrescriptionServiceImpl.java

Entity: Prescription.java

Repo: PrescriptionRepository.java

Flow:

Create prescription: POST /api/prescriptions (doctor role required)

Get by id: GET /api/prescriptions/{id}

Rules:

patientId, appointmentId, medication, dosage, instructions required.

doctorUsername is derived from auth header context.

issuedAt auto-set on persist (@PrePersist).

Note:

Service has getPrescriptionsByPatientId(...) in PrescriptionService.java, but controller does not currently expose a matching endpoint.

#### **Workflow 5: Patient Report Viewing**

Controller: DoctorPatientRecordController.java

Current impl: InMemoryDoctorPatientRecordService.java

Flow:

Doctor requests patient reports:

GET /api/doctors/{doctorUsername}/patients/{patientId}/reports

Important current status:

Access checks are enforced.

Response is currently an empty list placeholder (no patient-service integration yet).

Practical End-to-End Sequence (Doctor Journey)

Register/login via auth + gateway.

Upsert own doctor profile.

Define availability slots (replace/add/delete).

Fetch appointment requests.

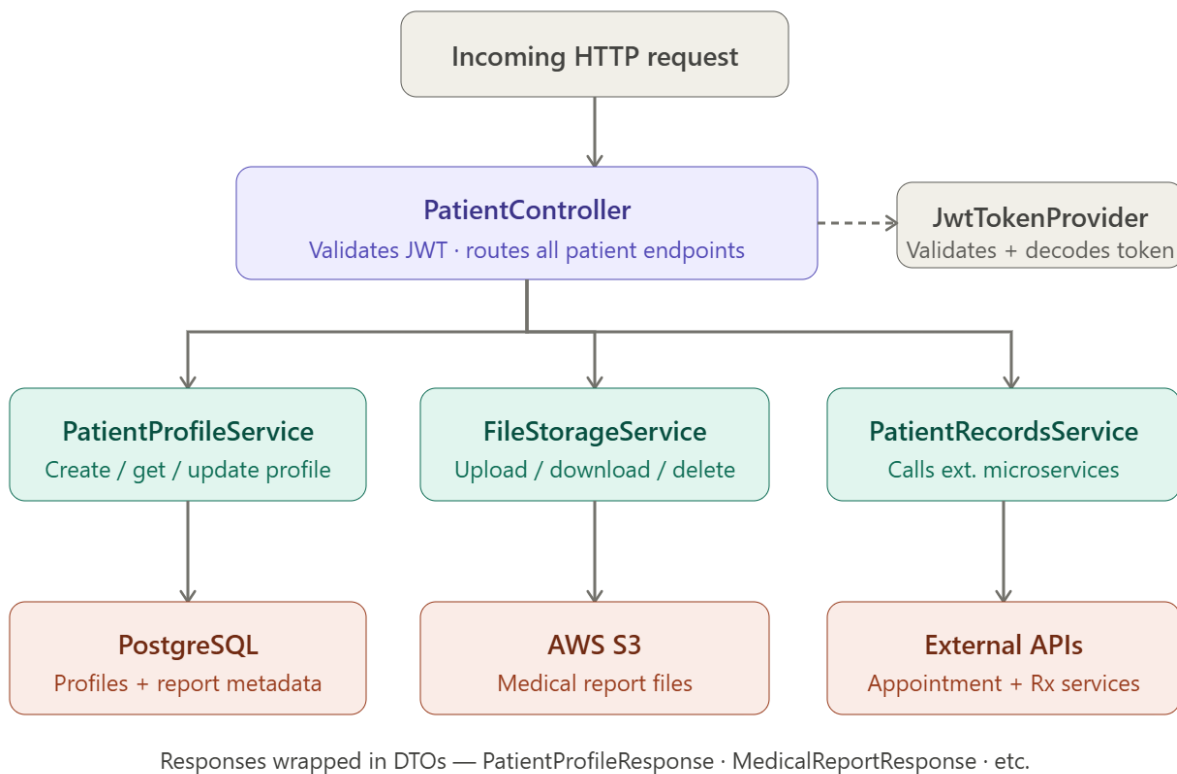
Accept/reject requests.

Issue prescription after consultation.

View patient reports/history before issuing care decisions.

## **Patient Service Workflow (IT23168336 C S B D Chandrathilaka)**

### **Workflow Overview**



The Patient Service is the central hub for managing patient data throughout their healthcare journey. It handles patient registration, profile management, medical history tracking, prescription management, and medical report storage.

### Step-by-Step Patient Service Flow

**Phase 1: Patient Registration & Authentication** Patient Registration → Auth Service issues JWT token  
Token Validation → Patient includes Bearer token in Authorization header  
Profile Creation → POST /api/patients creates initial patient profile  
Response → PatientProfileResponse returns patientId for future use

### Phase 2: Profile Management

Patient Request → GET /api/patients/me with JWT token  
JWT Extraction → Extracts username from token  
Profile Lookup → Queries database for patient record  
Response → Returns complete profile with all demographics

**Phase 3: Profile Updates** Patient Update → PUT /api/patients/me with new data Authorization Check → Validates JWT and extracts username Database Update → PatientProfileService updates all fields Audit Log → Change written to patient\_audit\_log table Notification → Patient notified of profile update Response → Returns updated profile

**Phase 4: Medical History Retrieval** Patient Request → GET /api/patients/{id}/history Authorization → Ensures patient can only access own data Service Call → Calls Appointment Service /api/appointments/patient/{patientId} Aggregation → Combines appointment data with diagnosis and notes Response → Returns list of past appointments with medical details

**Phase 5: Prescription Management** Patient Request → GET /api/patients/{id}/prescriptions Authorization → Validates ownership Service Call → Calls Doctor Service to retrieve prescriptions Formatting → Formats as PatientPrescriptionResponse objects Response → Returns all active and past prescriptions

**Phase 6: Medical Report Upload** Patient Upload → POST /api/patients/{id}/reports with multipart file Authorization → Confirms patient identity File Processing → FileStorageService uploads file to S3 Metadata Storage → Saves filename, date, description to DB DB Entry → Creates MedicalReport entity linked to patient Response → Returns downloadUrl for future access

**Phase 7: Medical Report Listing** Patient Request → GET /api/patients/{id}/reports DB Query → Retrieves all reports for patient URL Generation → Creates download URL for each report Response → Returns list of MedicalReportResponse objects

**Phase 8: Medical Report Download** Patient Request → GET /api/patients/{id}/reports/{reportId}/download Authorization → Validates ownership of report File Retrieval → FileStorageService fetches file bytes from S3 MIME Detection → Determines correct content type (PDF, image, etc.) Response → Streams file as ByteArrayResource to browser

**Phase 9: Medical Report Deletion** Patient Request → DELETE /api/patients/{id}/reports/{reportId} Authorization → Validates ownership File Deletion → Removes object from S3 DB Cleanup → Deletes MedicalReport entity Response → 204 No Content

**Phase 10: AI Symptom Checker** Patient Input → POST /api/symptoms/analyze with Bearer token JWT Validation → Auth Gateway validates authentication Cache Check → Hashes symptoms and checks Redis for existing result Cache Hit → Returns cached analysis immediately Cache Miss →



SymptomAnalysisService builds detailed Gemini prompt Gemini Call → Sends prompt to Google Gemini API Response Parsing → Extracts conditions, specialists, warnings, next steps Confidence Score → Calculates LOW / MEDIUM / HIGH based on symptom clarity Cache Store → Stores result with 24h TTL Response → Returns SymptomAnalysisResponse to frontend

# Authentication & Security Mechanisms

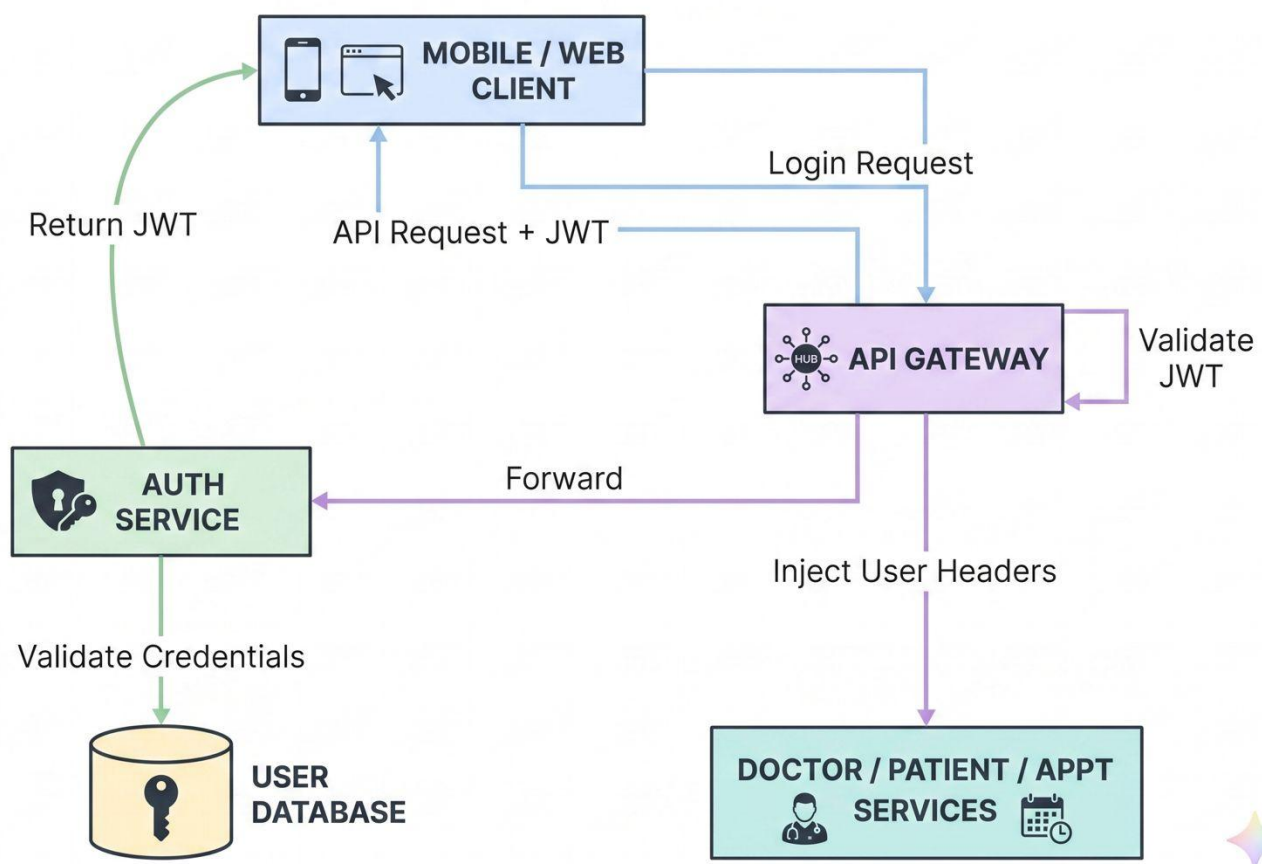
This project uses a modern, stateless microservices security architecture centered around JWT (JSON Web Tokens) and a central API Gateway.

Below is a detailed breakdown of the authentication and security mechanisms implemented in the project.

## 1. Overall Security Architecture

The security model follows a "Gatekeeper" pattern. The API Gateway acts as the single entry point and security enforcer, while the Auth Service handles the identity management.

### Authentication & API Gateway Flow



## 2. Core Security Components

### A. Centralized Authentication (Auth Service)

Located in backend/auth-service, this service is responsible for:

- **Registration & Login**: Public endpoints (`/api/auth/register`, `/api/auth/login`) to onboard and authenticate users.
- **Password Hashing**: Uses **BCrypt** hashing algorithm to securely store passwords in the database.

- **Token Issuance:** Upon successful login, the JwtTokenProvider generates a JWT signed with a secret key (using the HS512 algorithm).
- **Payload:** The JWT contains:
  - sub: The username/identity of the user.
  - role/authorities: The user's permissions (e.g., ROLE\_PATIENT, ROLE\_DOCTOR).
  - exp: Expiration timestamp (defaulted to 24 hours).

## B. API Gateway Gatekeeper (API Gateway)

Located in backend/api-gateway, this service enforces security for the entire system:

- **JWT Validation:** The JwtAuthenticationFilter intercepts every incoming request. It validates the token's signature, integrity, and expiration.
- **Identity Propagation:** Once a token is validated, the gateway extracts the user's identity and roles. It then injects these into new HTTP headers:
  - X-User-Id: The authenticated username.
  - X-User-Role: The user's specific role.
- **Downstream Trust:** Because all traffic must pass through the gateway, downstream microservices (like doctor-service or appointment-service) can trust these X-User-\* headers for business logic and authorization.

## C. Spring Security Integration

Both the Gateway and Auth Service utilize **Spring Security 6.x** to manage the filter chains and security configurations:

- **Stateless Policy:** Configured to SessionCreationPolicy.STATELESS, meaning no server-side sessions are maintained.
- **CORS Management:** Configured in SecurityConfig.java to handle cross-origin requests from the frontend.
- **CSRF Protection:** Disabled by design, as JWT-based stateless APIs are inherently protected against standard CSRF attacks that target session cookies.

## 3. Service-to-Service Security

- **Isolation:** In a production environment (as seen in the docker-compose.yml), the microservices communicate over an internal network.
- **Header-Based Authorization:** Individual services use the headers injected by the gateway to determine if a user has the right to perform an action (e.g., a Patient can only see their own appointments).

#### 4. Technical Specifications

| Feature                     | Implementation                                 |
|-----------------------------|------------------------------------------------|
| <b>Authentication Type</b>  | Stateless JWT (Bearer Token)                   |
| <b>Algorithm</b>            | HS512 (HMAC with SHA-512)                      |
| <b>Password Encoding</b>    | BCrypt                                         |
| <b>Identity Propagation</b> | HTTP Header Injection (X-User-Id, X-User-Role) |
| <b>Framework</b>            | Spring Security 6.x                            |

## Individual contribution

| Student Number | Contribution                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
|----------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| IT23158986     | <p>Implemented Appointment Service (Search doctors by specialty, name, availability, date, Book, modify, and cancel appointments, Real-time appointment status tracking, Send appointment events to Notification Service via message queue, Appointment history for both patients and doctors)</p> <p>Implemented Telemedicine Service (Integrate a third-party video API: Agora, Generate secure, time-limited session tokens per consultation, Create and manage video rooms (start, join, end), Link each video session to a confirmed appointment, Handle session state (waiting, active, completed), Log session duration and completion status)</p> |
| IT23166042     | <p>Implemented the doctor-service backend and its associated frontend components for the healthcare platform. On the backend, I developed doctor profile management, authentication for doctor registration and login, availability scheduling, and prescription handling.</p> <p>On the frontend, I implemented the doctor-facing interface for profile updates, availability management, appointment-related workflows, and prescription operations, delivering an integrated end-to-end experience for doctor users.</p>                                                                                                                               |
| IT23168336     | <p>Built Patient Service (Port 8082) - Implemented comprehensive patient profile management system with registration, profile updates, medical history tracking, prescription management, and secure medical report upload/download with full JWT authorization and role-based access control.</p> <p>Built AI Symptom Checker Service (Port 8087) - Integrated Google Gemini API for intelligent symptom analysis, providing preliminary health suggestions with probable conditions, recommended doctor specialties, warning signs detection, and comprehensive medical disclaimers ensuring patient safety.</p>                                        |
| IT23292604     | <p>Payment service: Implemented complete payment processing pipeline with Stripe/PayPal integration, transaction management, refund handling, and admin dashboard for tracking all payments and revenue analytics.</p> <p>Notification service : Developed multi-channel notification system supporting EMAIL, SMS, PUSH, and IN_APP messaging with HTML email templates, notification persistence, delivery tracking, and complete audit trails for compliance.</p>                                                                                                                                                                                      |

# Appendix

## Appointment Service

```
package com.SE73.appointment_service.service.impl;

import com.SE73.appointment_service.client.DoctorClient;
import com.SE73.appointment_service.dto.AppointmentRequest;
import com.SE73.appointment_service.dto.AppointmentResponse;
import com.SE73.appointment_service.dto.AppointmentStatusUpdateRequest;
import com.SE73.appointment_service.enums.AppointmentMode;
import com.SE73.appointment_service.enums.AppointmentStatus;
import com.SE73.appointment_service.enums.PaymentStatus;
import com.SE73.appointment_service.exception.AppointmentNotFoundException;
import com.SE73.appointment_service.exception.InvalidAppointmentStatusException;
import com.SE73.appointment_service.exception.SlotAlreadyBookedException;
import com.SE73.appointment_service.model.Appointment;
import com.SE73.appointment_service.repository.AppointmentRepository;
import com.SE73.appointment_service.service.AppointmentService;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import java.util.Arrays;
import java.util.List;
import java.util.UUID;
import java.util.stream.Collectors;

/**
 * Implementation of {@link AppointmentService} containing the core business logic
```

```
* for appointment management in the healthcare platform.
```

```
*/
```

```
@Service
```

```
public class AppointmentServiceImpl implements AppointmentService {
```

```
    private static final Logger logger = LoggerFactory.getLogger(AppointmentServiceImpl.class);
```

```
    /** Valid status values that can be set via the status update endpoint */
```

```
    private static final List<String> VALID_UPDATE_STATUSES = Arrays.asList(
```

```
        AppointmentStatus.CONFIRMED.name(),
```

```
        AppointmentStatus.CANCELLED.name(),
```

```
        AppointmentStatus.COMPLETED.name()
```

```
    );
```

```
    private final AppointmentRepository appointmentRepository;
```

```
    private final DoctorClient doctorClient;
```

```
    // Constructor injection (no field injection)
```

```
    public AppointmentServiceImpl(AppointmentRepository appointmentRepository, DoctorClient  
doctorClient) {
```

```
        this.appointmentRepository = appointmentRepository;
```

```
        this.doctorClient = doctorClient;
```

```
    }
```

```
    // -----
```

```
    // Create
```

```
    // -----
```

```
    /**
```

```
        * Creates a new appointment.
```

```
* Validates that the requested slot has not already been booked before persisting.  
*  
* @param request the booking request DTO  
* @return the created appointment response  
* @throws SlotAlreadyBookedException if the slotId is already associated with an appointment  
*/
```

```
@Override
```

```
@Transactional
```

```
public AppointmentResponse createAppointment(AppointmentRequest request) {  
    logger.info("Creating appointment for patient '{}' with slot '{}'",  
        request.getPatientId(), request.getSlotId());  
  
    // Prevent double-booking: check if slot is already taken by a non-cancelled appointment  
    if (appointmentRepository.existsBySlotIdAndStatusNot(request.getSlotId(),  
AppointmentStatus.CANCELLED)) {  
        logger.warn("Slot '{}' is already booked", request.getSlotId());  
        throw new SlotAlreadyBookedException(  
            "Slot " + request.getSlotId() + " is already booked. Please choose a different slot."  
        );  
    }  
  
    // Validate and parse appointmentMode  
    AppointmentMode mode = parseAppointmentMode(request.getAppointmentMode());  
  
    // Build and populate the Appointment entity  
    Appointment appointment = new Appointment();  
    appointment.setPatientId(request.getPatientId());  
    appointment.setDoctorUsername(request.getDoctorUsername());  
    appointment.setSlotId(request.getSlotId());  
    appointment.setAppointmentDateTime(request.getAppointmentDateTime());
```



```

    appointment.setAppointmentMode(mode);
    appointment.setHospital(request.getHospital());
    appointment.setNotes(request.getNotes());
    // Status and paymentStatus default to PENDING (set in entity field defaults)

    Appointment saved = appointmentRepository.save(appointment);
    logger.info("Appointment created with ID '{}'", saved.getAppointmentsId());

    // Mark the slot as inactive (booked) in the doctor service
    doctorClient.updateSlotStatus(saved.getSlotId(), false);

    return mapToResponse(saved);
}

// -----
// Read
// -----

@Override
@Transactional(readOnly = true)
public AppointmentResponse getAppointmentsById(UUID id) {
    logger.debug("Fetching appointments with ID '{}'", id);
    Appointment appointment = findOrThrow(id);
    return mapToResponse(appointment);
}

@Override
@Transactional(readOnly = true)
public List<AppointmentResponse> getAppointmentsByPatientId(String patientId) {
    logger.debug("Fetching appointments for patient '{}'", patientId);

```

```
return appointmentRepository.findByPatientId(patientId)
    .stream()
    .map(this::mapToResponse)
    .collect(Collectors.toList());
}
```

@Override

@Transactional(readOnly = true)

```
public List<AppointmentResponse> getAppointmentsByDoctorUsername(String doctorUsername) {
    logger.debug("Fetching appointments for doctor '{}'", doctorUsername);
    return appointmentRepository.findByDoctorUsername(doctorUsername)
        .stream()
        .map(this::mapToResponse)
        .collect(Collectors.toList());
}
```

@Override

@Transactional(readOnly = true)

```
public List<AppointmentResponse> getAllAppointments() {
    logger.debug("Fetching all appointments");
    return appointmentRepository.findAll()
        .stream()
        .map(this::mapToResponse)
        .collect(Collectors.toList());
}
```

// -----

// Update

// -----

```

/**
 * Updates the status (and optionally notes) of an existing appointment.
 *
 * @param id    the appointment UUID
 * @param request the status update request
 * @return the updated appointment response
 * @throws AppointmentNotFoundException if the appointment does not exist
 * @throws InvalidAppointmentStatusException if the requested status is not a valid update value
 */

@Override
@Transactional

public AppointmentResponse updateAppointmentStatus(UUID id, AppointmentStatusUpdateRequest
request) {
    logger.info("Updating status of appointment '{}' to '{}'", id, request.getStatus());

    Appointment appointment = findOrThrow(id);

    // Validate that the new status is an allowed transition value
    String newStatusStr = request.getStatus().toUpperCase();
    if (!VALID_UPDATE_STATUSES.contains(newStatusStr)) {
        throw new InvalidAppointmentStatusException(
            "Invalid status value: " + request.getStatus() +
            ". Valid values are: " + String.join(", ", VALID_UPDATE_STATUSES)
        );
    }

    AppointmentStatus newStatus = AppointmentStatus.valueOf(newStatusStr);
    appointment.setStatus(newStatus);

    // Update notes if provided

```

```

    if (request.getNotes() != null && !request.getNotes().isBlank()) {
        appointment.setNotes(request.getNotes());
    }

    Appointment updated = appointmentRepository.save(appointment);
    logger.info("Appointment '{}' status updated to '{}'", id, newStatus);

    if (newStatus == AppointmentStatus.CANCELLED) {
        doctorClient.updateSlotStatus(updated.getSlotId(), true);
    }

    return mapToResponse(updated);
}

/**
 * Cancels an existing appointment by setting its status to CANCELLED.
 *
 * @param id the appointment UUID
 * @return the cancelled appointment response
 * @throws AppointmentNotFoundException if the appointment does not exist
 */
@Override
@Transactional
public AppointmentResponse cancelAppointment(UUID id) {
    logger.info("Cancelling appointment '{}'", id);

    Appointment appointment = findOrThrow(id);
    appointment.setStatus(AppointmentStatus.CANCELLED);

    Appointment cancelled = appointmentRepository.save(appointment);

```

```

        logger.info("Appointment '{}' has been cancelled", id);

        doctorClient.updateSlotStatus(cancelled.getSlotId(), true);

        return mapToResponse(cancelled);
    }

/**
 * Updates the payment status of an existing appointment.
 *
 * @param id the appointment UUID
 * @param status the new payment status
 */
@Override
@Transactional
public void updatePaymentStatus(UUID id, PaymentStatus status) {
    logger.info("Updating payment status of appointment '{}' to '{}'", id, status);

    Appointment appointment = findOrThrow(id);
    appointment.setPaymentStatus(status);

    appointmentRepository.save(appointment);
    logger.info("Appointment '{}' payment status updated to '{}'", id, status);
}

// -----
// Private Helpers
// -----

/**

```

```
* Looks up an appointment by ID; throws {@link AppointmentNotFoundException} if absent.
```

```
*/
```

```
private Appointment findOrThrow(UUID id) {
```

```
    return appointmentRepository.findById(id)
```

```
        .orElseThrow(() -> new AppointmentNotFoundException(
```

```
            "Appointment not found with ID: " + id
```

```
        ));
```

```
}
```

```
/**
```

```
* Parses an appointment mode string, throwing a meaningful error if invalid.
```

```
*/
```

```
private AppointmentMode parseAppointmentMode(String mode) {
```

```
    try {
```

```
        return AppointmentMode.valueOf(mode.toUpperCase());
```

```
    } catch (IllegalArgumentException e) {
```

```
        throw new InvalidAppointmentStatusException(
```

```
            "Invalid appointment mode: '" + mode + "'. Valid values are: VIRTUAL, PHYSICAL"
```

```
        );
```

```
    }
```

```
}
```

```
/**
```

```
* Maps an {@link Appointment} entity to an {@link AppointmentResponse} DTO.
```

```
*/
```

```
private AppointmentResponse mapToResponse(Appointment appointment) {
```

```
    AppointmentResponse response = new AppointmentResponse();
```

```
    response.setAppointmentId(appointment.getAppointmentId());
```

```
    response.setPatientId(appointment.getPatientId());
```

```
    response.setDoctorUsername(appointment.getDoctorUsername());
```

```
response.setSlotId(appointment.getSlotId());
response.setAppointmentDateTime(appointment.getAppointmentDateTime());
response.setAppointmentMode(appointment.getAppointmentMode() != null
    ? appointment.getAppointmentMode().name() : null);
response.setHospital(appointment.getHospital());
response.setStatus(appointment.getStatus() != null
    ? appointment.getStatus().name() : null);
response.setPrice(appointment.getPrice());
response.setPaymentStatus(appointment.getPaymentStatus() != null
    ? appointment.getPaymentStatus().name() : null);
response.setNotes(appointment.getNotes());
response.setCreatedAt(appointment.getCreatedAt());
response.setUpdatedAt(appointment.getUpdatedAt());
return response;
}
```

## Doctor Service

@RestController

@RequestMapping("/api/doctors/availability")

public class DoctorAvailabilityController {

private final DoctorAvailabilityService availabilityService;

public DoctorAvailabilityController(DoctorAvailabilityService availabilityService) {

this.availabilityService = availabilityService;

}

@GetMapping("/all")

public ResponseEntity<?> getAllDoctorAvailability(

@RequestHeader(value = "X-User-Id", required = false) String userId,

```
@RequestHeader(value = "X-User-Role", required = false) String userRole
) {
try {
validatePatientOrDoctorContext(userId, userRole);

List<DoctorAvailabilityResponse> response = availabilityService.getAllActiveSlots()
.stream()
.map(this::toDoctorAvailabilityResponse)
.toList();
return ResponseEntity.ok(response);
} catch (IllegalArgumentException e) {
return ResponseEntity.status(HttpStatus.FORBIDDEN).body(new ErrorResponse(e.getMessage()));
}
}
```

```
@GetMapping
public ResponseEntity<?> getDoctorAvailability(
    @RequestHeader(value = "X-User-Id", required = false) String userId,
    @RequestHeader(value = "X-User-Role", required = false) String userRole
) {
try {
String doctorUsername = validateDoctorContext(userId, userRole);
List<AvailabilitySlotResponse> response = availabilityService.getAvailability(doctorUsername)
.stream()
.map(this::toResponse)
.toList();
return ResponseEntity.ok(response);
} catch (IllegalArgumentException e) {
return ResponseEntity.status(HttpStatus.FORBIDDEN).body(new ErrorResponse(e.getMessage()));
}
}
```



### @PutMapping

```
public ResponseEntity<?> replaceDoctorAvailability(
    @RequestHeader(value = "X-User-Id", required = false) String userId,
    @RequestHeader(value = "X-User-Role", required = false) String userRole,
    @Valid @RequestBody SetDoctorAvailabilityRequest request
) {
    try {
        String doctorUsername = validateDoctorContext(userId, userRole);

        List<AvailabilitySlotResponse> response = availabilityService.replaceAvailability(doctorUsername,
            request.getSlots())
            .stream()
            .map(this::toResponse)
            .toList();
        return ResponseEntity.ok(response);
    } catch (IllegalArgumentException e) {
        return ResponseEntity.status(HttpStatus.BAD_REQUEST).body(new ErrorResponse(e.getMessage()));
    }
}
```

### @PostMapping("/slots")

```
public ResponseEntity<?> addSlot(
    @RequestHeader(value = "X-User-Id", required = false) String userId,
    @RequestHeader(value = "X-User-Role", required = false) String userRole,
    @Valid @RequestBody AvailabilitySlotRequest request
) {
    try {
        String doctorUsername = validateDoctorContext(userId, userRole);
        AvailabilitySlotResponse response = toResponse(availabilityService.addSlot(doctorUsername, request));
        return ResponseEntity.status(HttpStatus.CREATED).body(response);
    }
}
```

```
} catch (IllegalArgumentException e) {  
return ResponseEntity.status(HttpStatus.BAD_REQUEST).body(new ErrorResponse(e.getMessage()));  
}  
}
```

```
@DeleteMapping("/slots/{slotId}")  
public ResponseEntity<?> deleteSlot(  
    @RequestHeader(value = "X-User-Id", required = false) String userId,  
    @RequestHeader(value = "X-User-Role", required = false) String userRole,  
    @PathVariable UUID slotId  
) {  
    try {  
        String doctorUsername = validateDoctorContext(userId, userRole);  
        availabilityService.deleteSlot(doctorUsername, slotId);  
        return ResponseEntity.noContent().build();  
    } catch (IllegalArgumentException e) {  
        return ResponseEntity.status(HttpStatus.BAD_REQUEST).body(new ErrorResponse(e.getMessage()));  
    }  
}
```

```
private String validateDoctorContext(String userId, String userRole) {  
    if (!StringUtils.hasText(userId)) {  
        throw new IllegalArgumentException("Missing authenticated user context");  
    }  
  
    if (!"ROLE_DOCTOR".equalsIgnoreCase(userRole)) {  
        throw new IllegalArgumentException("Only doctors can manage availability");  
    }  
  
    return userId;  
}
```

```
}

private void validatePatientOrDoctorContext(String userId, String userRole) {
    if (!StringUtils.hasText(userId)) {
        throw new IllegalArgumentException("Missing authenticated user context");
    }

    if (!"ROLE_DOCTOR".equalsIgnoreCase(userRole) && !"ROLE_PATIENT".equalsIgnoreCase(userRole))
    {
        throw new IllegalArgumentException("Only doctors or patients can access availability information");
    }
}

private AvailabilitySlotResponse toResponse(DoctorAvailabilitySlot slot) {
    return new AvailabilitySlotResponse(
        slot.getId(),
        slot.getDayOfWeek(),
        slot.getStartTime(),
        slot.getEndTime(),
        slot.isActive()
    );
}

private DoctorAvailabilityResponse toDoctorAvailabilityResponse(DoctorAvailabilitySlot slot) {
    return new DoctorAvailabilityResponse(
        slot.getId(),
        slot.getDoctorUsername(),
        slot.getDayOfWeek(),
        slot.getStartTime(),
        slot.getEndTime()
    );
}
```

```
);  
}  
  
private record ErrorResponse(String message) {  
}  
}
```

## Patient service

```
package com.se73.patient_service.controller;  
  
import com.se73.patient_service.dto.CreatePatientProfileRequest;  
import com.se73.patient_service.dto.PatientAppointmentHistoryResponse;  
import com.se73.patient_service.dto.PatientProfileResponse;  
import com.se73.patient_service.dto.PatientPrescriptionResponse;  
import com.se73.patient_service.dto.UpdatePatientProfileRequest;  
import com.se73.patient_service.dto.MedicalReportResponse;  
import com.se73.patient_service.model.PatientProfile;  
import com.se73.patient_service.model.MedicalReport;  
import com.se73.patient_service.security.JwtTokenProvider;  
import com.se73.patient_service.service.PatientProfileService;  
import com.se73.patient_service.service.FileStorageService;  
import com.se73.patient_service.service.PatientRecordsService;  
import com.se73.patient_service.repository.MedicalReportRepository;  
import jakarta.validation.Valid;  
import org.springframework.http.HttpStatus;  
import org.springframework.http.HttpHeaders;  
import org.springframework.http.MediaType;  
import org.springframework.http.ResponseEntity;  
import org.springframework.core.io.ByteArrayResource;
```

```
import org.springframework.web.bind.annotation.*;

import org.springframework.web.multipart.MultipartFile;


import java.util.List;

import java.util.stream.Collectors;


@RestController
@RequestMapping("/api/patients")
@CrossOrigin(origins = "**")
public class PatientController {


    private final PatientProfileService patientProfileService;
    private final JwtTokenProvider jwtTokenProvider;
    private final FileStorageService fileStorageService;
    private final MedicalReportRepository medicalReportRepository;
    private final PatientRecordsService patientRecordsService;


    public PatientController(PatientProfileService patientProfileService, JwtTokenProvider
jwtTokenProvider,
                           FileStorageService fileStorageService, MedicalReportRepository medicalReportRepository,
                           PatientRecordsService patientRecordsService) {
        this.patientProfileService = patientProfileService;
        this.jwtTokenProvider = jwtTokenProvider;
        this.fileStorageService = fileStorageService;
        this.medicalReportRepository = medicalReportRepository;
        this.patientRecordsService = patientRecordsService;
    }


    @PostMapping
    public ResponseEntity<?> createProfile(
```

```

    @RequestHeader("Authorization") String authHeader,
    @Valid @RequestBody CreatePatientProfileRequest request
) {
    try {
        String username = extractUsername(authHeader);
        PatientProfile profile = patientProfileService.createProfile(username, request);
        return ResponseEntity.status(HttpStatus.CREATED).body(new PatientProfileResponse(profile));
    } catch (IllegalArgumentException e) {
        return ResponseEntity.status(HttpStatus.UNAUTHORIZED).body(new
ErrorResponse(e.getMessage()));
    } catch (RuntimeException e) {
        return ResponseEntity.status(HttpStatus.BAD_REQUEST).body(new
ErrorResponse(e.getMessage()));
    }
}

@GetMapping("/me")
public ResponseEntity<?> getMyProfile(@RequestHeader("Authorization") String authHeader) {
    try {
        String username = extractUsername(authHeader);
        PatientProfile profile = patientProfileService.getProfile(username);
        return ResponseEntity.ok(new PatientProfileResponse(profile));
    } catch (IllegalArgumentException e) {
        return ResponseEntity.status(HttpStatus.UNAUTHORIZED).body(new
ErrorResponse(e.getMessage()));
    } catch (RuntimeException e) {
        return ResponseEntity.status(HttpStatus.NOT_FOUND).body(new ErrorResponse(e.getMessage()));
    }
}

@PostMapping("/me")

```

```

public ResponseEntity<?> updateMyProfile(
    @RequestHeader("Authorization") String authHeader,
    @Valid @RequestBody UpdatePatientProfileRequest request
) {
    try {
        String username = extractUsername(authHeader);
        PatientProfile profile = patientProfileService.updateProfile(username, request);
        return ResponseEntity.ok(new PatientProfileResponse(profile));
    } catch (IllegalArgumentException e) {
        return ResponseEntity.status(HttpStatus.UNAUTHORIZED).body(new
ErrorResponse(e.getMessage()));
    } catch (RuntimeException e) {
        return ResponseEntity.status(HttpStatus.BAD_REQUEST).body(new
ErrorResponse(e.getMessage()));
    }
}

```

```

@GetMapping("/{id}/history")
public ResponseEntity<?> getPatientHistory(
    @RequestHeader("Authorization") String authHeader,
    @PathVariable Long id
) {
    try {
        PatientProfile patient = getAuthorizedPatient(authHeader, id);
        List<PatientAppointmentHistoryResponse> history =
patientRecordsService.getPatientHistory(patient);
        return ResponseEntity.ok(history);
    } catch (IllegalArgumentException e) {
        return ResponseEntity.status(HttpStatus.UNAUTHORIZED).body(new
ErrorResponse(e.getMessage()));
    } catch (SecurityException e) {

```

```

        return ResponseEntity.status(HttpStatus.FORBIDDEN).body(new ErrorResponse(e.getMessage()));
    } catch (RuntimeException e) {
        return ResponseEntity.status(HttpStatus.BAD_GATEWAY).body(new
ErrorResponse(e.getMessage()));
    }
}

```

```

@GetMapping("/{id}/prescriptions")
public ResponseEntity<?> getPatientPrescriptions(
    @RequestHeader("Authorization") String authHeader,
    @PathVariable Long id
) {
    try {
        PatientProfile patient = getAuthorizedPatient(authHeader, id);
        List<PatientPrescriptionResponse> prescriptions =
patientRecordsService.getPatientPrescriptions(patient);
        return ResponseEntity.ok(prescriptions);
    } catch (IllegalArgumentException e) {
        return ResponseEntity.status(HttpStatus.UNAUTHORIZED).body(new
ErrorResponse(e.getMessage()));
    } catch (SecurityException e) {
        return ResponseEntity.status(HttpStatus.FORBIDDEN).body(new ErrorResponse(e.getMessage()));
    } catch (RuntimeException e) {
        return ResponseEntity.status(HttpStatus.BAD_GATEWAY).body(new
ErrorResponse(e.getMessage()));
    }
}

```

```

@PostMapping("/{id}/reports")
public ResponseEntity<?> uploadReport(
    @RequestHeader("Authorization") String authHeader,

```



```

    @PathVariable Long id,
    @RequestParam("file") MultipartFile file,
    @RequestParam(value = "description", required = false) String description
) {
    try {
        getAuthorizedPatient(authHeader, id);

        MedicalReport report = fileStorageService.uploadFile(id, file, description);
        return ResponseEntity.status(HttpStatus.CREATED).body(new MedicalReportResponse(report,
"/api/patients/" + id + "/reports/" + report.getId() + "/download"));
    } catch (IllegalArgumentException e) {
        return ResponseEntity.status(HttpStatus.UNAUTHORIZED).body(new
ErrorResponse(e.getMessage()));
    } catch (SecurityException e) {
        return ResponseEntity.status(HttpStatus.FORBIDDEN).body(new ErrorResponse(e.getMessage()));
    } catch (RuntimeException e) {
        return ResponseEntity.status(HttpStatus.BAD_REQUEST).body(new
ErrorResponse(e.getMessage()));
    }
}

@GetMapping("/{id}/reports")
public ResponseEntity<?> getReports(
    @RequestHeader("Authorization") String authHeader,
    @PathVariable Long id
) {
    try {
        getAuthorizedPatient(authHeader, id);

        List<MedicalReport> reports = medicalReportRepository.findByPatientId(id);
        List<MedicalReportResponse> responses = reports.stream()

```

```

        .map(r -> new MedicalReportResponse(r, "/api/patients/" + id + "/reports/" + r.getId() +
"/download"))
        .collect(Collectors.toList());
    return ResponseEntity.ok(responses);
} catch (IllegalArgumentException e) {
    return ResponseEntity.status(HttpStatus.UNAUTHORIZED).body(new
ErrorResponse(e.getMessage()));
} catch (SecurityException e) {
    return ResponseEntity.status(HttpStatus.FORBIDDEN).body(new ErrorResponse(e.getMessage()));
} catch (RuntimeException e) {
    return ResponseEntity.status(HttpStatus.NOT_FOUND).body(new ErrorResponse(e.getMessage()));
}
}

```

```

@GetMapping("/{id}/reports/{reportId}")
public ResponseEntity<?> getReport(
    @RequestHeader("Authorization") String authHeader,
    @PathVariable Long id,
    @PathVariable Long reportId
) {
    try {
        getAuthorizedPatient(authHeader, id);

        MedicalReport report = medicalReportRepository.findByIdAndPatientId(reportId, id)
            .orElseThrow(() -> new RuntimeException("Report not found"));

        return ResponseEntity.ok(new MedicalReportResponse(report, "/api/patients/" + id + "/reports/" +
report.getId() + "/download"));
    } catch (IllegalArgumentException e) {
        return ResponseEntity.status(HttpStatus.UNAUTHORIZED).body(new
ErrorResponse(e.getMessage()));
    } catch (SecurityException e) {

```

```

        return ResponseEntity.status(HttpStatus.FORBIDDEN).body(new ErrorResponse(e.getMessage()));
    } catch (RuntimeException e) {
        return ResponseEntity.status(HttpStatus.NOT_FOUND).body(new ErrorResponse(e.getMessage()));
    }
}

@GetMapping("/{id}/reports/{reportId}/download")
public ResponseEntity<?> downloadReport(
    @RequestHeader("Authorization") String authHeader,
    @PathVariable Long id,
    @PathVariable Long reportId
) {
    try {
        getAuthorizedPatient(authHeader, id);

        FileStorageService.DownloadedFile downloadedFile = fileStorageService.downloadFile(reportId,
id);

        return ResponseEntity.ok()
            .header(HttpHeaders.CONTENT_DISPOSITION, "inline; filename=\"" +
downloadedFile.fileName() + "\"")
            .contentType(MediaType.parseMediaType(downloadedFile.contentType()))
            .body(new ByteArrayResource(downloadedFile.content()));
    } catch (IllegalArgumentException e) {
        return ResponseEntity.status(HttpStatus.UNAUTHORIZED).body(new
ErrorResponse(e.getMessage()));
    } catch (SecurityException e) {
        return ResponseEntity.status(HttpStatus.FORBIDDEN).body(new ErrorResponse(e.getMessage()));
    } catch (RuntimeException e) {
        return ResponseEntity.status(HttpStatus.NOT_FOUND).body(new ErrorResponse(e.getMessage()));
    }
}
}

```

```
@DeleteMapping("/{id}/reports/{reportId}")
public ResponseEntity<?> deleteReport(
    @RequestHeader("Authorization") String authHeader,
    @PathVariable Long id,
    @PathVariable Long reportId
) {
    try {
        getAuthorizedPatient(authHeader, id);

        fileStorageService.deleteFile(reportId, id);
        return ResponseEntity.noContent().build();
    } catch (IllegalArgumentException e) {
        return ResponseEntity.status(HttpStatus.UNAUTHORIZED).body(new
ErrorResponse(e.getMessage()));
    } catch (SecurityException e) {
        return ResponseEntity.status(HttpStatus.FORBIDDEN).body(new ErrorResponse(e.getMessage()));
    } catch (RuntimeException e) {
        return ResponseEntity.status(HttpStatus.BAD_REQUEST).body(new
ErrorResponse(e.getMessage()));
    }
}

private PatientProfile getAuthorizedPatient(String authHeader, Long id) {
    String username = extractUsername(authHeader);
    PatientProfile patient = patientProfileService.getProfile(username);
    if (!patient.getId().equals(id)) {
        throw new SecurityException("You can only access your own patient data");
    }
    return patient;
}
```

```
}

private String extractUsername(String authHeader) {
    if (authHeader == null || !authHeader.startsWith("Bearer ")) {
        throw new IllegalArgumentException("Invalid token format");
    }

    String token = authHeader.substring(7);
    if (!jwtTokenProvider.validateToken(token)) {
        throw new IllegalArgumentException("Invalid token");
    }

    return jwtTokenProvider.getUsernameFromToken(token);
}

public static class ErrorResponse {
    private final String message;

    public ErrorResponse(String message) {
        this.message = message;
    }

    public String getMessage() {
        return message;
    }
}
}
```

## Telemedicine Service

```
package com.se73.telemedicine_service.service;

import com.se73.telemedicine_service.exception.ApiException;
import io.agora.media.RtcTokenBuilder2;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.stereotype.Service;

import java.util.regex.Pattern;

@Service
public class AgoraTokenService {

    private static final Pattern AGORA_APP_ID_PATTERN = Pattern.compile("[A-Za-z0-9]{32}$");

    @Value("${agora.app-id}")
    private String agoraAppId;

    @Value("${agora.app-certificate}")
    private String agoraAppCertificate;

    @Value("${agora.token-ttl-seconds:3600}")
    private int defaultTtlSeconds;

    public String getAgoraAppId() {
        return sanitizeAndValidateAppId();
    }

    public int getDefaultTtlSeconds() {
        return defaultTtlSeconds;
    }
}
```

```
public String generateSessionToken(String roomName, int ttlSeconds) {  
    String appId = sanitizeAndValidateAppId();  
    String appCertificate = sanitizeAndValidateAppCertificate();  
  
    RtcTokenBuilder2 tokenBuilder = new RtcTokenBuilder2();  
  
    // Use 0 as UID which means the token is valid for any UID  
    // This matches the frontend expectation where the frontend generates its own UID  
    return tokenBuilder.buildTokenWithUid(  
        appId,  
        appCertificate,  
        roomName,  
        0,  
        RtcTokenBuilder2.Role.ROLE_PUBLISHER,  
        ttlSeconds,  
        ttlSeconds  
    );  
}
```

```
private String sanitizeAndValidateAppId() {  
    String normalized = agoraAppId == null ? "" : agoraAppId.trim();  
    if (normalized.isEmpty() || "demo-app-id".equalsIgnoreCase(normalized)) {  
        throw new ApiException("AGORA_APP_ID is not configured. Set a valid Agora App ID in  
backend/.env and restart services.");  
    }  
  
    if (!AGORA_APP_ID_PATTERN.matcher(normalized).matches()) {  
        throw new ApiException("AGORA_APP_ID format is invalid. Expected a 32-character Agora App  
ID.");  
    }  
}
```

```
    }

    return normalized;
}

private String sanitizeAndValidateAppCertificate() {
    String normalized = agoraAppCertificate == null ? "" : agoraAppCertificate.trim();
    if (normalized.isEmpty() || "demo-app-certificate".equalsIgnoreCase(normalized)) {
        throw new ApiException("AGORA_APP_CERTIFICATE is not configured. Set a valid Agora App
Certificate in backend/.env and restart services.");
    }
    return normalized;
}

}
```

```
-----

package com.se73.telemedicine_service.service;

import com.se73.telemedicine_service.dto.SessionCreateRequest;
import com.se73.telemedicine_service.dto.SessionResponse;
import com.se73.telemedicine_service.dto.SessionTokenResponse;
import com.se73.telemedicine_service.exception.ApiException;
import com.se73.telemedicine_service.model.CompletionStatus;
import com.se73.telemedicine_service.model.SessionState;
import com.se73.telemedicine_service.model.TelemedicineSession;
import org.springframework.stereotype.Service;

import java.time.LocalDateTime;
import java.util.Map;
```



```
import java.util.UUID;

import java.util.concurrent.ConcurrentHashMap;

@Service

public class TelemedicineSessionService {

    private final Map<UUID, TelemedicineSession> sessions = new ConcurrentHashMap<>();

    private final AppointmentClient appointmentClient;

    private final AgoraTokenService agoraTokenService;

    public TelemedicineSessionService(AppointmentClient appointmentClient, AgoraTokenService
agoraTokenService) {

        this.appointmentClient = appointmentClient;

        this.agoraTokenService = agoraTokenService;

    }

    public SessionResponse createSession(SessionCreateRequest request) {

        appointmentClient.assertAppointmentConfirmed(request.getAppointmentId());

        UUID sessionId = UUID.randomUUID();

        String roomName = "consultation-" + request.getAppointmentId();

        TelemedicineSession session = TelemedicineSession.builder()

            .sessionId(sessionId)

            .appointmentId(request.getAppointmentId())

            .roomName(roomName)

            .state(SessionState.WAITING)

            .completionStatus(CompletionStatus.ONGOING)

            .createdAt(LocalDateTime.now())

            .build();

    }

}
```

```
    sessions.put(sessionId, session);  
    return toResponse(session);  
}
```

```
public SessionTokenResponse getSessionToken(UUID sessionId) {  
    TelemedicineSession session = getRequiredSession(sessionId);  
  
    if (session.getState() == SessionState.COMPLETED) {  
        throw new ApiException("Cannot issue token for completed session");  
    }  
  
    int ttl = agoraTokenService.getDefaultTtlSeconds();  
    String token = agoraTokenService.generateSessionToken(session.getRoomName(), ttl);  
  
    return SessionTokenResponse.builder()  
        .sessionId(session.getSessionId())  
        .roomName(session.getRoomName())  
        .agoraAppId(agoraTokenService.getAgoraAppId())  
        .token(token)  
        .expiresAt(LocalDateTime.now().plusSeconds(ttl))  
        .build();  
}
```

```
public SessionResponse startSession(UUID sessionId) {  
    TelemedicineSession session = getRequiredSession(sessionId);  
  
    if (session.getState() == SessionState.COMPLETED) {  
        throw new ApiException("Completed session cannot be started");  
    }  
}
```

```
if (session.getState() == SessionState.WAITING) {  
    session.setState(SessionState.ACTIVE);  
    session.setStartedAt(LocalDateTime.now());  
}  
  
return toResponse(session);  
}  
  
public SessionResponse endSession(UUID sessionId) {  
    TelemedicineSession session = getRequiredSession(sessionId);  
  
    if (session.getState() == SessionState.COMPLETED) {  
        return toResponse(session);  
    }  
  
    session.setState(SessionState.COMPLETED);  
    session.setEndedAt(LocalDateTime.now());  
  
    if (session.getStartedAt() != null) {  
        long duration = java.time.Duration.between(session.getStartedAt(),  
session.getEndedAt()).getSeconds();  
        session.setDurationSeconds(Math.max(duration, 0));  
        session.setCompletionStatus(CompletionStatus.COMPLETED_NORMALLY);  
    } else {  
        session.setDurationSeconds(0L);  
        session.setCompletionStatus(CompletionStatus.ENDED_BEFORE_START);  
    }  
  
    return toResponse(session);  
}
```

```
}
```

```
public SessionResponse getSessionStatus(UUID sessionId) {  
    return toResponse(getRequiredSession(sessionId));  
}
```

```
public SessionResponse getSessionByAppointment(UUID appointmentId) {  
    return sessions.values().stream()  
        .filter(s -> appointmentId.equals(s.getAppointmentId()))  
        .findFirst()  
        .map(this::toResponse)  
        .orElseThrow(() -> new ApiException("No telemedicine session found for appointment: " +  
appointmentId));  
}
```

```
private TelemedicineSession getRequiredSession(UUID sessionId) {  
    TelemedicineSession session = sessions.get(sessionId);  
    if (session == null) {  
        throw new ApiException("Session not found: " + sessionId);  
    }  
    return session;  
}
```

```
private SessionResponse toResponse(TelemedicineSession session) {  
    return SessionResponse.builder()  
        .sessionId(session.getSessionId())  
        .appointmentId(session.getAppointmentId())  
        .roomName(session.getRoomName())  
        .state(session.getState())  
        .completionStatus(session.getCompletionStatus())
```

```
        .createdAt(session.getCreatedAt())
        .startedAt(session.getStartedAt())
        .endedAt(session.getEndedAt())
        .durationSeconds(session.getDurationSeconds())
        .build();
    }
}
```

## Payment service

```
package com.se73.payment_service.service;
```

```
import com.se73.payment_service.dto.PaymentInitiateRequest;
import com.se73.payment_service.dto.PaymentResponse;
import com.se73.payment_service.entity.PaymentTransaction;
import com.se73.payment_service.exception.PaymentException;
import com.stripe.exception.StripeException;
import com.stripe.model.PaymentIntent;
import com.stripe.param.PaymentIntentCreateParams;
import lombok.extern.slf4j.Slf4j;
import org.springframework.stereotype.Service;
```

```
import java.math.BigDecimal;
```

```
@Service
```

```
@Slf4j
```

```
public class StripePaymentService {
```

```
    public PaymentIntent createPaymentIntent(PaymentInitiateRequest request, PaymentTransaction
transaction)
```

```
        throws StripeException {
```

```
// Convert amount to cents (Stripe works with cents)
long amountInCents = request.getAmount()
    .multiply(new BigDecimal(100))
    .longValue();

PaymentIntentCreateParams params = PaymentIntentCreateParams.builder()
    .setAmount(amountInCents)
    .setCurrency(request.getCurrency().toLowerCase())
    .setDescription(request.getDescription() + " - Appointment: " + request.getAppointmentId())
    .putMetadata("appointmentId", request.getAppointmentId().toString())
    .putMetadata("patientId", request.getPatientId())
    .putMetadata("transactionId", transaction.getId().toString())
    .setAutomaticPaymentMethods(
        PaymentIntentCreateParams.AutomaticPaymentMethods.builder()
            .setEnabled(true)
            .build()
    )
    .build();

return PaymentIntent.create(params);
}

public PaymentIntent retrievePaymentIntent(String paymentIntentId) throws StripeException {
    return PaymentIntent.retrieve(paymentIntentId);
}

public PaymentIntent confirmPaymentIntent(String paymentIntentId, String paymentMethodId)
    throws StripeException {
```

```
PaymentIntent intent = PaymentIntent.retrieve(paymentIntentId);

// For testing with test payment methods (pm_test_*), simulate success
if (paymentMethodId != null && paymentMethodId.startsWith("pm_test_")) {
    log.info("Test payment method detected: {}", simulating payment success", paymentMethodId);
    // Update intent to succeeded status for testing
    intent = intent.update(
        java.util.Collections.singletonMap("payment_method", paymentMethodId)
    );
    // For test cards, immediately mark as succeeded
    return intent;
} else if (paymentMethodId != null && !paymentMethodId.isEmpty()) {
    // For real payment methods, set it on the intent
    intent = intent.update(
        java.util.Collections.singletonMap("payment_method", paymentMethodId)
    );
    log.info("Payment method set for intent: {}", paymentIntentId);
}

return intent;
}

public void refundPaymentIntent(String paymentIntentId) throws StripeException {
    PaymentIntent intent = PaymentIntent.retrieve(paymentIntentId);
    if ("succeeded".equals(intent.getStatus())) {
        intent.cancel();
        log.info("Refund initiated for payment intent: {}", paymentIntentId);
    } else {
        throw new PaymentException("Cannot refund payment with status: " + intent.getStatus());
    }
}
```

```
}  
}
```

## Notification Service

```
package com.se73.notification_service.service;
```

```
import tools.jackson.core.type.TypeReference;  
import tools.jackson.databind.ObjectMapper;  
import com.se73.notification_service.entity.Notification;  
import lombok.extern.slf4j.Slf4j;  
import org.springframework.beans.factory.annotation.Qualifier;  
import org.springframework.stereotype.Service;  
import org.thymeleaf.TemplateEngine;  
import org.thymeleaf.context.Context;
```

```
import java.util.Map;
```

```
@Service
```

```
@Slf4j
```

```
public class EmailTemplateService {
```

```
    private final TemplateEngine templateEngine;
```

```
    private final ObjectMapper objectMapper;
```

```
    public EmailTemplateService(@Qualifier("emailTemplateEngine") TemplateEngine templateEngine,  
                               ObjectMapper objectMapper) {  
        this.templateEngine = templateEngine;  
        this.objectMapper = objectMapper;  
    }
```



// Renders the Thymeleaf HTML template for the given notification type; falls back to plain bodyText on error

```
public String renderForNotification(Notification notification) {
    String templateName = templateFor(notification.getType());
    Context ctx = new Context();

    ctx.setVariable("subject", notification.getSubject());
    ctx.setVariable("bodyText", notification.getBodyText());
    ctx.setVariable("recipientName", notification.getRecipientUserId());

    Map<String, Object> payload = parsePayload(notification.getPayloadJson());
    payload.forEach(ctx::setVariable);

    try {
        return templateEngine.process(templateName, ctx);
    } catch (Exception e) {
        log.warn("Template '{}' failed ({}), falling back to plain body", templateName, e.getMessage());
        return "<p>" + (notification.getBodyText() != null ? notification.getBodyText() : "") + "</p>";
    }
}

private String templateFor(Notification.Type type) {
    return switch (type) {
        case PAYMENT_CONFIRMED -> "payment-confirmed";
        case PAYMENT_FAILED -> "payment-failed";
        case APPOINTMENT_BOOKED -> "appointment-booked";
        default -> "generic";
    };
}
```

```
private Map<String, Object> parsePayload(String payloadJson) {  
    if (payloadJson == null || payloadJson.isBlank()) {  
        return Map.of();  
    }  
    try {  
        return objectMapper.readValue(payloadJson, new TypeReference<Map<String, Object>>() {});  
    } catch (Exception e) {  
        log.warn("Failed to parse notification payload JSON: {}", e.getMessage());  
        return Map.of();  
    }  
}
```

## AI Symptom Checker Service

```
package com.se73.ai_symptom_checker.service;  
  
import com.se73.ai_symptom_checker.dto.SymptomCheckRequest;  
import com.se73.ai_symptom_checker.dto.SymptomCheckResponse;  
import com.se73.ai_symptom_checker.dto.HealthCondition;  
import lombok.extern.slf4j.Slf4j;  
import org.springframework.beans.factory.annotation.Value;  
import org.springframework.stereotype.Service;  
import com.fasterxml.jackson.databind.ObjectMapper;  
import java.io.OutputStream;  
import java.net.HttpURLConnection;  
import java.net.URL;  
import java.nio.charset.StandardCharsets;  
import java.time.LocalDateTime;  
import java.time.format.DateTimeFormatter;
```

```
import java.util.Arrays;

import java.util.List;

import java.util.Locale;

import java.util.Scanner;


@Slf4j
@Service

public class SymptomAnalysisService {


    @Value("${gemini.api.key}")
    private String geminiApiKey;


    @Value("${gemini.api.model}")
    private String geminiModel;


    @Value("${symptom.mock.enabled:true}")
    private boolean mockEnabled;


    private final ObjectMapper objectMapper = new ObjectMapper();

    private static final String GEMINI_API_URL_TEMPLATE =
"https://generativelanguage.googleapis.com/v1beta/models/%s:generateContent";


    public SymptomCheckResponse analyzeSymptoms(SymptomCheckRequest request) {
        log.info("Analyzing symptoms: {}", request.getSymptoms());


        try {
            /*

            if (shouldUseMockResponse()) {

                log.warn("Using mock symptom analysis response (mockEnabled={}, apiKeyConfigured={})",
mockEnabled, isApiKeyConfigured());
```

```
        return buildMockResponse(request);
    }
    */

    String prompt = buildAnalysisPrompt(request);
    String geminiResponse = callGeminiAPI(prompt);

    if (geminiResponse == null || geminiResponse.isBlank()) {
        log.error("Gemini response was empty, check API key and model name");
        throw new RuntimeException("Gemini API returned an empty response. Verify if the model name
and API key are correct.");
    }

    return parseGeminiResponse(geminiResponse, request);
} catch (Exception e) {
    log.error("Error analyzing symptoms: {}", e.getMessage(), e);
    throw new RuntimeException("AI Analysis failed: " + e.getMessage(), e);
}
}

private boolean shouldUseMockResponse() {
    return mockEnabled || !isApiKeyConfigured();
}

private boolean isApiKeyConfigured() {
    return geminiApiKey != null
        && !geminiApiKey.isBlank()
        && !geminiApiKey.contains("your-gemini-api-key-here");
}
```

```
private String buildAnalysisPrompt(SymptomCheckRequest request) {  
    return String.format(  
        ""
```

You are a medical AI assistant. Analyze the following symptoms and provide preliminary health suggestions.

PATIENT INFORMATION:

- Symptoms: %s
- Duration: %s
- Severity: %s
- Age Group: %s
- Additional Info: %s

Please provide a response in JSON format with the following structure:

```
{  
  "conditions": [  
    {  
      "name": "Condition Name",  
      "probability": 75,  
      "description": "Brief description",  
      "characteristics": ["symptom1", "symptom2"],  
      "severity": "mild/moderate/severe"  
    }  
  ],  
  "recommendedSpecialties": ["Specialty 1", "Specialty 2"],  
  "warnings": ["Warning 1", "Warning 2"],  
  "nextSteps": "What the patient should do",  
  "confidence": "high/medium/low"  
}
```

### IMPORTANT RESTRICTIONS:

1. Only suggest top 3-5 most probable conditions
2. Do not diagnose - only provide preliminary suggestions
3. Always include a medical disclaimer
4. If any warning signs present, highlight urgently
5. Recommend seeing a doctor for proper diagnosis

Provide ONLY the JSON response, no additional text.

```
""",
String.join(", ", request.getSymptoms()),
request.getDuration(),
request.getSeverity(),
request.getAgeGroup(),
request.getAdditionalInfo() != null ? request.getAdditionalInfo() : "None"
);
}

private String callGeminiAPI(String prompt) {
    try {
        log.info("Calling Gemini API with model: {}", geminiModel);

        // Build request body using ObjectMapper for proper escaping
        var contents = List.of(
            java.util.Map.of("parts", List.of(
                java.util.Map.of("text", prompt)
            ))
        );

        String requestBody = objectMapper.writeValueAsString(java.util.Map.of("contents", contents));

        // Make HTTP POST request
```

```
String endpoint = String.format(GEMINI_API_URL_TEMPLATE, geminiModel);
URL url = new URL(endpoint + "?key=" + geminiApiKey);
URLConnection connection = (URLConnection) url.openConnection();
connection.setRequestMethod("POST");
connection.setRequestProperty("Content-Type", "application/json");
connection.setDoOutput(true);

// Send request
try (OutputStream os = connection.getOutputStream()) {
    byte[] input = requestBody.getBytes(StandardCharsets.UTF_8);
    os.write(input, 0, input.length);
}

// Read response
int responseCode = connection.getResponseCode();
log.info("Gemini API Response Code: {}", responseCode);

if (responseCode == 200) {
    String response = readResponseStream(connection.getInputStream());
    log.info("Gemini API Response received successfully");
    return extractTextFromGeminiResponse(response);
} else {
    // Read error response
    String errorResponse = readResponseStream(connection.getErrorStream());
    log.error("Gemini API Error Response Code: {} - {}", responseCode, errorResponse);
    throw new RuntimeException("Gemini API error (Code " + responseCode + "): " +
errorResponse);
}

} catch (Exception e) {
```

```

    log.error("Error calling Gemini API: {}", e.getMessage(), e);
    if (e instanceof RuntimeException) {
        throw (RuntimeException) e;
    }
    throw new RuntimeException("API call failed: " + e.getMessage());
}
}

```

```

private String readResponseStream(java.io.InputStream stream) throws Exception {
    if (stream == null) {
        return "";
    }
    try (Scanner scanner = new Scanner(stream, StandardCharsets.UTF_8)) {
        scanner.useDelimiter("\\A");
        return scanner.hasNext() ? scanner.next() : "";
    }
}

```

```

private String extractTextFromGeminiResponse(String jsonResponse) throws Exception {
    // Parse Gemini response and extract the text content
    com.fasterxml.jackson.databind.JsonNode root = objectMapper.readTree(jsonResponse);
    com.fasterxml.jackson.databind.JsonNode candidates = root.get("candidates");

    if (candidates != null && candidates.isArray() && candidates.size() > 0) {
        com.fasterxml.jackson.databind.JsonNode firstCandidate = candidates.get(0);
        com.fasterxml.jackson.databind.JsonNode content = firstCandidate.get("content");
        if (content != null) {
            com.fasterxml.jackson.databind.JsonNode parts = content.get("parts");
            if (parts != null && parts.isArray() && parts.size() > 0) {
                com.fasterxml.jackson.databind.JsonNode textNode = parts.get(0).get("text");
            }
        }
    }
}

```



```
        if (textNode != null) {
            return textNode.asText();
        }
    }
}

log.error("Could not extract text from Gemini response: {}", jsonResponse);
return null;
}

private SymptomCheckResponse parseGeminiResponse(String aiText, SymptomCheckRequest request) {
    try {
        log.debug("Parsing AI response: {}", aiText);

        // Try to find a JSON block in the response
        String jsonToParse = aiText;

        // Check for markdown code blocks
        if (aiText.contains("```")) {
            int start = aiText.indexOf("```json");
            if (start == -1) {
                start = aiText.indexOf("```");
            } else {
                start += 7; // Skip ```json
            }

            if (start != -1) {
                if (aiText.indexOf("```", start) == -1 && start < 7) {
```

```

        start += 3; // Skip ``` if it wasn't ```json
    }

    int end = aiText.indexOf("```", start);
    if (end != -1) {
        jsonToParse = aiText.substring(start, end).trim();
    }
}
} else {
    // If no code blocks, try to find the first '{' and last '}'
    int start = aiText.indexOf('{');
    int end = aiText.lastIndexOf('}');
    if (start != -1 && end != -1 && end > start) {
        jsonToParse = aiText.substring(start, end + 1).trim();
    }
}
}

```

```

SymptomCheckResponse response = objectMapper.readValue(jsonToParse,
SymptomCheckResponse.class);

```

```

    response.setAnalysisTimestamp(LocalDateTime.now().format(DateTimeFormatter.ISO_LOCAL_D
ATE_TIME));

```

```

    response.setDisclaimer("This is a preliminary AI-based analysis and should NOT be used as a
substitute for professional medical advice. Always consult with a qualified healthcare provider for accurate
diagnosis and treatment.");

```

```

    return response;

```

```

} catch (Exception e) {

```

```

    log.error("Error parsing AI response: {}. Original text: {}", e.getMessage(), aiText, e);

```

```

    throw new RuntimeException("Failed to parse AI response: " + e.getMessage(), e);

```

```

}

```

```

}

```

```
private SymptomCheckResponse buildMockResponse(SymptomCheckRequest request) {  
    String combinedSymptoms = String.join(" ", request.getSymptoms()).toLowerCase(Locale.ROOT);  
    List<HealthCondition> conditions;  
    List<String> specialties;  
    List<String> warnings;  
    String nextSteps;  
  
    if (containsAny(combinedSymptoms, "fever", "cough", "sore throat", "runny nose")) {  
        conditions = Arrays.asList(  
            new HealthCondition("Viral Upper Respiratory Infection", 78, "Common viral illness affecting  
nose and throat", Arrays.asList("fever", "cough", "sore throat"), "mild"),  
            new HealthCondition("Influenza-like Illness", 62, "May cause fever, body aches, and fatigue",  
Arrays.asList("fever", "fatigue", "body pain"), "moderate"),  
            new HealthCondition("Seasonal Allergy with Infection", 35, "Allergy symptoms with possible  
secondary irritation", Arrays.asList("runny nose", "sneezing", "mild cough"), "mild")  
        );  
        specialties = Arrays.asList("General Physician", "Pulmonologist");  
        warnings = Arrays.asList("Seek urgent care if breathing difficulty develops", "High fever above  
103F requires immediate medical attention");  
        nextSteps = "Hydrate well, rest, monitor fever every 6-8 hours, and consult a doctor if symptoms  
worsen after 48 hours.";  
    } else if (containsAny(combinedSymptoms, "headache", "dizziness", "migraine")) {  
        conditions = Arrays.asList(  
            new HealthCondition("Tension Headache", 71, "Often linked with stress, posture, or poor  
sleep", Arrays.asList("headache", "neck tightness"), "mild"),  
            new HealthCondition("Migraine Episode", 55, "Recurring headache often with light  
sensitivity", Arrays.asList("headache", "nausea", "light sensitivity"), "moderate"),  
            new HealthCondition("Dehydration-related Headache", 42, "Low fluid intake can trigger  
dizziness and headache", Arrays.asList("dizziness", "dry mouth", "fatigue"), "mild")  
        );  
        specialties = Arrays.asList("General Physician", "Neurologist");  
        warnings = Arrays.asList("Sudden severe headache needs emergency assessment", "Seek care if  
weakness, speech issues, or vision changes occur");  
    }  
}
```

```

        nextSteps = "Increase hydration, rest in a dark environment, and book a doctor consultation if
headaches persist or recur frequently.";
    } else {
        conditions = Arrays.asList(
            new HealthCondition("General Viral Syndrome", 58, "A non-specific viral condition with self-
limited symptoms", request.getSymptoms(), "mild"),
            new HealthCondition("Stress-related Somatic Symptoms", 41, "Physical symptoms that may
worsen during stress", Arrays.asList("fatigue", "sleep disturbance", "body discomfort"), "mild"),
            new HealthCondition("Early Inflammatory Condition", 33, "May need clinical examination and
basic lab tests", request.getSymptoms(), "moderate")
        );
        specialties = Arrays.asList("General Physician");
        warnings = Arrays.asList("Visit emergency care for chest pain, breathing difficulty, or confusion",
"Persistent symptoms beyond one week require clinical review");
        nextSteps = "Track symptoms for 24-48 hours and consult a doctor for proper diagnosis and
treatment planning.";
    }

    SymptomCheckResponse response = new SymptomCheckResponse();
    response.setConditions(conditions);
    response.setRecommendedSpecialties(specialties);
    response.setWarnings(warnings);
    response.setNextSteps(nextSteps);
    response.setConfidence("medium");
    response.setAnalysisTimestamp(LocalDateTime.now().format(DateTimeFormatter.ISO_LOCAL_DAT
E_TIME));
    response.setDisclaimer("Demo mock response: This is preliminary guidance only and not a diagnosis.
Please consult a qualified healthcare professional.");
    return response;
}

private boolean containsAny(String source, String... keywords) {

```

```
    for (String keyword : keywords) {  
        if (source.contains(keyword)) {  
            return true;  
        }  
    }  
    return false;  
}  
  
}
```